

Array.prototype.reduce()



Línea base Ampliamente disponible



El **reduce()** método de **Array** instancias ejecuta una función de devolución de llamada "reductora" proporcionada por el usuario en cada elemento del array, en orden, pasando el valor de retorno del cálculo del elemento anterior. El resultado final de ejecutar la función reductora en todos los elementos del array es un único valor.

La primera vez que se ejecuta la función de devolución de llamada, no se devuelve ningún valor del cálculo anterior. Si se proporciona, se puede usar un valor inicial en su lugar. De lo contrario, se usa el elemento del array en el índice 0 como valor inicial y la iteración comienza desde el siguiente elemento (índice 1 en lugar del índice 0).

Pruébalo

Demostración en JavaScript: Array.prototype.reduce()

```
1 const array = [1, 2, 3, 4];
2
3 // 0 + 1 + 2 + 3 + 4
4 const initialValue = 0;
5 const sumWithInitial = array.reduce(
6   (accumulator, currentValue) => accumulator + currentValue,
7   initialValue,
8 );
9
10 console.log(sumWithInitial);
11 // Expected output: 10
12
```

Sintaxis

JS

```
reduce(callbackFn)
reduce(callbackFn, initialValue)
```

Parámetros

callbackFn

Una función que se ejecuta para cada elemento del array. Su valor de retorno se convierte en el valor del `accumulator` parámetro en la siguiente invocación de `callbackFn`. Para la última invocación, el valor de retorno se convierte en el valor de retorno de `reduce()`. La función se llama con los siguientes argumentos:

accumulator

El valor resultante de la llamada anterior a `callbackFn`. En la primera llamada, su valor es `initialValue` si se especifica este último; de lo contrario, su valor es `array[0]`.

currentValue

El valor del elemento actual. En la primera llamada, su valor es `array[0]` si `initialValue` se especifica; de lo contrario, su valor es `array[1]`.

currentIndex

La posición del índice `currentValue` en el array. En la primera llamada, su valor es `0` si `initialValue` se especifica, de lo contrario `1`.

array

`reduce()` Se recurrió al sistema de arreglos.

initialValue Opcional

Un valor que `accumulator` se inicializa la primera vez que se llama a la función de devolución de llamada. Si `initialValue` se especifica, `callbackFn` comienza a ejecutarse con el primer valor del array como `currentValue`. Si *no* `initialValue` se especifica, se inicializa con el primer valor del array y comienza a ejecutarse con el segundo valor del array como `.` En este caso, si el array está vacío (de modo que no hay un primer valor que devolver como `.`), se genera un error. `accumulator callbackFn currentValue accumulator`

Valor de retorno

El valor que resulta de ejecutar la función de devolución de llamada "reductor" hasta su finalización en todo el array.

Excepciones

`TypeError`

Se lanza una excepción si el array no contiene elementos y `initialValue` no se proporciona.

Descripción

Este `reduce()` método es [iterativo](#). Ejecuta una función de devolución de llamada "reductora" sobre todos los elementos del array, en orden ascendente de índice, y los acumula en un único valor. En cada iteración, el valor de

retorno `callbackFn` se pasa `callbackFn` nuevamente a la siguiente llamada como `accumulator`. El valor final de `accumulator` (que es el valor devuelto por `callbackFn` en la última iteración del array) se convierte en el valor de retorno de `reduce()`. Consulta la sección [de métodos iterativos](#) para obtener más información sobre cómo funcionan estos métodos en general.

`callbackFn` Se invoca únicamente para los índices de matriz que tienen valores asignados. No se invoca para las ranuras vacías en [matrices dispersas](#).

A diferencia de otros [métodos iterativos](#), `reduce()` no acepta un `thisArg` argumento. `callbackFn` Siempre se llama con `undefined` como `this`, que se sustituye por `globalThis` si `callbackFn` no es estricto.

`reduce()` es un concepto central en [la programación funcional](#), donde no es posible mutar ningún valor, por lo que para acumular todos los valores en un array, se debe devolver un nuevo valor acumulador en cada iteración. Esta convención se propaga a JavaScript `reduce()`: se deben usar métodos [de propagación](#) u otros métodos de copia cuando sea posible para crear nuevos arrays y objetos como acumulador, en lugar de mutar el existente. Si decides mutar el acumulador en lugar de copiarlo, recuerda devolver el objeto modificado en la función de devolución de llamada, o la siguiente iteración recibirá `undefined`. Sin embargo, ten en cuenta que copiar el acumulador puede a su vez provocar un mayor uso de memoria y un rendimiento degradado; consulta [Cuándo no usar reduce\(\)](#) para obtener más detalles. En tales casos, para evitar un rendimiento deficiente y un código ilegible, es mejor usar un `for` bucle en su lugar.

El `reduce()` método es [genérico](#). Solo espera que el `this` valor tenga una `length` propiedad y propiedades con clave entera.

Casos límite

Si el array solo tiene un elemento (independientemente de la posición) y no `initialValue` se proporciona ninguno, o si `initialValue` se proporciona pero el array está vacío, se devolverá el valor único *sin* llamar a `callbackFn`.

Si `initialValue` se proporciona y el array no está vacío, el método `reduce` siempre invocará la función de devolución de llamada comenzando en el índice 0.

Si `initialValue` no se proporciona, el método `reduce` actuará de manera diferente para matrices con longitud mayor que 1, igual a 1 y 0, como se muestra en el siguiente ejemplo:

JS

```
const getMax = (a, b) => Math.max(a, b);

// callback is invoked for each element in the array starting at index 0
[1, 100].reduce(getMax, 50); // 100
[50].reduce(getMax, 10); // 50

// callback is invoked once for element at index 1
[1, 100].reduce(getMax); // 100

// callback is not invoked
[50].reduce(getMax); // 50
[].reduce(getMax, 1); // 1

[].reduce(getMax); // TypeError
```

Ejemplos

Cómo funciona reduce() sin un valor inicial

El siguiente código muestra lo que sucede si llamamos a la función `reduce()` con un array y sin valor inicial.

JS

```
const array = [15, 16, 17, 18, 19];

function reducer(accumulator, currentValue, index) {
  const returns = accumulator + currentValue;
  console.log(
    `accumulator: ${accumulator}, currentValue: ${currentValue}, index: ${index}, returns: ${returns}`,
  );
  return returns;
}

array.reduce(reducer);
```

La función de devolución de llamada se invocaría cuatro veces, y los argumentos y valores de retorno en cada llamada serían los siguientes:

	accumulator	currentValue	index	Valor de retorno
Primera llamada	15	16	1	31
Segunda llamada	31	17	2	48
Tercera llamada	48	18	3	66

	accumulator	currentValue	index	Valor de retorno
Cuarta llamada	66	19	4	85

El `array` parámetro nunca cambia durante el proceso; siempre es `[15, 16, 17, 18, 19]`. El valor devuelto por `reduce()` sería el de la última invocación de la función de devolución de llamada (`85`).

Cómo funciona `reduce()` con un valor inicial

Aquí reducimos el mismo array usando el mismo algoritmo, pero con un `initialValue` valor `10` pasado como segundo argumento a `reduce()` :

JS

```
[15, 16, 17, 18, 19].reduce(  
  (accumulator, currentValue) => accumulator + currentValue,  
  10,  
);
```

La función de devolución de llamada se invocaría cinco veces, y los argumentos y valores de retorno en cada llamada serían los siguientes:

	accumulator	currentValue	index	Valor de retorno
Primera llamada	10	15	0	25
Segunda llamada	25	16	1	41
Tercera llamada	41	17	2	58
Cuarta llamada	58	18	3	76
Quinta llamada	76	19	4	95

El valor devuelto `reduce()` en este caso sería `95`.

Suma de valores en un array de objetos

Para sumar los valores contenidos en una matriz de objetos, debe **proporcionar** un `initialValue`, de modo que cada elemento pase por su función.

JS

```
const objects = [{ x: 1 }, { x: 2 }, { x: 3 }];  
const sum = objects.reduce(  
  (accumulator, currentValue) => accumulator + currentValue.x,  
  0,
```

```
);  
  
console.log(sum); // 6
```

Tubería secuencial funcional

La `pipe` función recibe una secuencia de funciones y devuelve una nueva función. Cuando se llama a la nueva función con un argumento, la secuencia de funciones se ejecuta en orden, recibiendo cada una el valor de retorno de la función anterior.

JS

```
const pipe =  
  (...functions) =>  
  (initialValue) =>  
    functions.reduce((acc, fn) => fn(acc), initialValue);  
  
// Building blocks to use for composition  
const double = (x) => 2 * x;  
const triple = (x) => 3 * x;  
const quadruple = (x) => 4 * x;  
  
// Composed functions for multiplication of specific values  
const multiply6 = pipe(double, triple);  
const multiply9 = pipe(triple, triple);  
const multiply16 = pipe(quadruple, quadruple);  
const multiply24 = pipe(double, triple, quadruple);  
  
// Usage  
multiply6(6); // 36  
multiply9(9); // 81  
multiply16(16); // 256  
multiply24(10); // 240
```

Promesas en ejecución en secuencia

La [secuenciación de promesas](#) es esencialmente una canalización de funciones como la que se demostró en la sección anterior, pero realizada de forma asíncrona.

JS

```
// Compare this with pipe: fn(acc) is changed to acc.then(fn),  
// and initialValue is ensured to be a promise  
const asyncPipe =  
  (...functions) =>  
  (initialValue) =>
```

```
functions.reduce((acc, fn) => acc.then(fn), Promise.resolve(initialValue));

// Building blocks to use for composition
const p1 = async (a) => a * 5;
const p2 = async (a) => a * 2;
// The composed functions can also return non-promises, because the values are
// all eventually wrapped in promises
const f3 = (a) => a * 3;
const p4 = async (a) => a * 4;

asyncPipe(p1, p2, f3, p4)(10).then(console.log); // 1200
```

`asyncPipe` También se puede implementar usando `async` / `await`, lo que demuestra mejor su similitud con `pipe`:

JS

```
const asyncPipe =
  (...functions) =>
  (initialValue) =>
    functions.reduce(async (acc, fn) => fn(await acc), initialValue);
```

Uso de reduce() con arreglos dispersos

`reduce()` omite los elementos que faltan en los arreglos dispersos, pero no omite `undefined` los valores.

JS

```
console.log([1, 2, , 4].reduce((a, b) => a + b)); // 7
console.log([1, 2, undefined, 4].reduce((a, b) => a + b)); // NaN
```

Llamar a reduce() en objetos que no son arrays

El `reduce()` método lee la `length` propiedad de `this` y luego accede a cada propiedad cuya clave es un entero no negativo menor que `length`.

JS

```
const arrayLike = {
  length: 3,
  0: 2,
  1: 3,
  2: 4,
  3: 99, // ignored by reduce() since length is 3
};
```

```
console.log(Array.prototype.reduce.call(arrayLike, (x, y) => x + y));  
// 9
```

Cuándo no usar reduce()

Las funciones de orden superior multipropósito como `reduce()` pueden ser potentes pero a veces difíciles de entender, especialmente para los desarrolladores de JavaScript con menos experiencia. Si el código se vuelve más claro al usar otros métodos de array, los desarrolladores deben sopesar la pérdida de legibilidad frente a los demás beneficios de usar `reduce()`.

Tenga en cuenta que `reduce()` siempre es equivalente a un `for...of` bucle, excepto que en lugar de modificar una variable en el ámbito superior, ahora devolvemos el nuevo valor en cada iteración:

JS

```
const val = array.reduce((acc, cur) => update(acc, cur), initialValue);  
  
// Is equivalent to:  
let val = initialValue;  
for (const cur of array) {  
  val = update(val, cur);  
}
```

Como se indicó anteriormente, la razón por la que la gente podría querer usarlo `reduce()` es para imitar las prácticas de programación funcional de datos inmutables. Por lo tanto, los desarrolladores que defienden la inmutabilidad del acumulador a menudo copian todo el acumulador en cada iteración, de esta manera:

JS

```
const names = ["Alice", "Bob", "Tiff", "Bruce", "Alice"];  
const countedNames = names.reduce((allNames, name) => {  
  const currCount = Object.hasOwn(allNames, name) ? allNames[name] : 0;  
  return {  
    ...allNames,  
    [name]: currCount + 1,  
  };  
}, {});
```



Este código tiene un rendimiento deficiente, ya que cada iteración tiene que copiar el `allNames` objeto completo, que puede ser grande, dependiendo de cuántos nombres únicos haya. Este código tiene $O(N^2)$ un rendimiento en el peor de los casos, donde N es la longitud de `names`.

Una mejor alternativa es *modificar* el `allNames` objeto en cada iteración. Sin embargo, si `allNames` se modifica de todos modos, es posible que desee convertirlo `reduce()` en un `for` bucle, lo cual es mucho más claro:

JS

```
const names = ["Alice", "Bob", "Tiff", "Bruce", "Alice"];
const countedNames = names.reduce((allNames, name) => {
  const currCount = allNames[name] ?? 0;
  allNames[name] = currCount + 1;
  // return allNames, otherwise the next iteration receives undefined
  return allNames;
}, Object.create(null));
```



JS

```
const names = ["Alice", "Bob", "Tiff", "Bruce", "Alice"];
const countedNames = Object.create(null);
for (const name of names) {
  const currCount = countedNames[name] ?? 0;
  countedNames[name] = currCount + 1;
}
```



Por lo tanto, si su acumulador es un array o un objeto y está copiando el array o el objeto en cada iteración, podría introducir accidentalmente una complejidad cuadrática en su código, lo que provocaría una rápida degradación del rendimiento con grandes cantidades de datos. Esto ha ocurrido en código real; véase, por ejemplo, « [Cómo hacer que Tanstack Table sea 1000 veces más rápido con un cambio de una sola línea](#) » [↗](#).

Algunos de los casos de uso aceptables `reduce()` se mencionan más arriba (en particular, la suma de un array, la secuenciación de promesas y la canalización de funciones). Existen otros casos en los que `reduce()` hay mejores alternativas.

- Aplanar una matriz de matrices. Use `flat()` en su lugar.

JS

```
const flattened = array.reduce((acc, cur) => acc.concat(cur), []);
```



JS

```
const flattened = array.flat();
```



- Agrupar objetos por una propiedad. Utilice `Object.groupBy()` en su lugar.

JS

```
const groups = array.reduce((acc, obj) => {  
  const key = obj.name;  
  const curGroup = acc[key] ?? [];  
  return { ...acc, [key]: [...curGroup, obj] };  
}, {});
```



JS

```
const groups = Object.groupBy(array, (obj) => obj.name);
```



- Concatenar matrices contenidas en una matriz de objetos. Utilice `flatMap()` en su lugar.

JS

```
const friends = [  
  { name: "Anna", books: ["Bible", "Harry Potter"] },  
  { name: "Bob", books: ["War and peace", "Romeo and Juliet"] },  
  { name: "Alice", books: ["The Lord of the Rings", "The Shining"] },  
];  
const allBooks = friends.reduce((acc, cur) => [...acc, ...cur.books], []);
```



JS

```
const allBooks = friends.flatMap((person) => person.books);
```



- Eliminar elementos duplicados en una matriz. Use `Set` y `Array.from()` en su lugar.

JS

```
const uniqArray = array.reduce(  
  (acc, cur) => (acc.includes(cur) ? acc : [...acc, cur]),  
  [],  
);
```



JS

```
const uniqArray = Array.from(new Set(array));
```



- Eliminar o agregar elementos en una matriz. Use `flatMap()` en su lugar.

JS

```
// Takes an array of numbers and splits perfect squares into its square roots
const roots = array.reduce((acc, cur) => {
  if (cur < 0) return acc;
  const root = Math.sqrt(cur);
  if (Number.isInteger(root)) return [...acc, root, root];
  return [...acc, cur];
}, []);
```



JS

```
const roots = array.flatMap((val) => {
  if (val < 0) return [];
  const root = Math.sqrt(val);
  if (Number.isInteger(root)) return [root, root];
  return [val];
});
```



Si solo estás eliminando elementos de una matriz, también puedes usar `filter()`.

- Para buscar elementos o comprobar si cumplen una condición, utilice `find()` and `findIndex()` o `some()` `every()` and `.`. Estos métodos tienen la ventaja adicional de que devuelven el resultado en cuanto se conoce, sin necesidad de iterar sobre todo el array.

JS

```
const allEven = array.reduce((acc, cur) => acc && cur % 2 === 0, true);
```



JS

```
const allEven = array.every((val) => val % 2 === 0);
```



En los casos en que `reduce()` sea la mejor opción, la documentación y la denominación semántica de las variables pueden ayudar a mitigar los inconvenientes de legibilidad.

Presupuesto

Especificación

Especificación del lenguaje ECMAScript® 2027 # sec-array.prototype.reduce

Compatibilidad con navegadores

[Informar de problemas con estos datos de compatibilidad](#) • [Ver datos en GitHub](#)

	Escritorio					Móvil							Otro		
	Cromo	Borde	Firefox	Ópera	Safari	Chrome para Android	Firefox para Android	Opera Android	Safari en iOS	Internet de Samsung	WebView Android	WebView en iOS	Bollo	Deno	Node.js
reduce	✓ 3	✓ 12	✓ 3	✓ 10.5	✓ 4	✓ 18	✓ 4	✓ 14	✓ 3.2	✓ 1	✓ 4.4	✓ 3.2	✓ 1	✓ 1	✓ 0,10

✓ Soporte completo

Véase también

- Relleno de poliéster `Array.prototype.reduce` en `core-js`
- `es-shims polyfill` de `Array.prototype.reduce`
- Guía de colecciones indexadas
- `Array`
- `Array.prototype.map()`
- `Array.prototype.flat()`
- `Array.prototype.flatMap()`
- `Array.prototype.reduceRight()`
- `TypedArray.prototype.reduce()`
- `Object.groupBy()`
- `Map.groupBy()`



Tu plan para una mejor internet.